

E-Commerce Database System — Final Project Report

Course: CSE 303 Lab - Database Management (Summer 2026)

Institution: Independent University, Bangladesh (IUB)

Prepared By:

- **Asiful Islam** (ID: 2420197, Section: 5)
 - **Sakib Mohammad Ome** (ID: 2220867, Section: 4)
 - **Shadman Samin Shahriar** (ID: 2211120, Section: 2)
-

Project Goals and System Overview

1. Executive Summary & Project Context

The primary objective of this project is the end-to-end design, implementation, academic verification, and cloud deployment of a production-grade **E-Commerce Database System**. Developed to satisfy the rigorous syllabus requirements of the **CSE 303 Lab (Database Management)** curriculum, the system resolves real-world enterprise database management challenges encountered by modern online retail platforms.

Online retail databases must handle concurrent transactions, maintain strict inventory consistency, enforce complex regulatory and operational constraints, and execute analytical queries for executive decision-making. This project bridges pure theoretical database theory (Relational Algebra, 1NF–3NF Normalization, B+ Tree Indexing algorithms, PL/pgSQL Triggers, and RAID Fault Tolerance) with modern web engineering practices.

2. Business Problem Statement

Legacy or simplified e-commerce software frequently suffers from catastrophic data management flaws, such as:

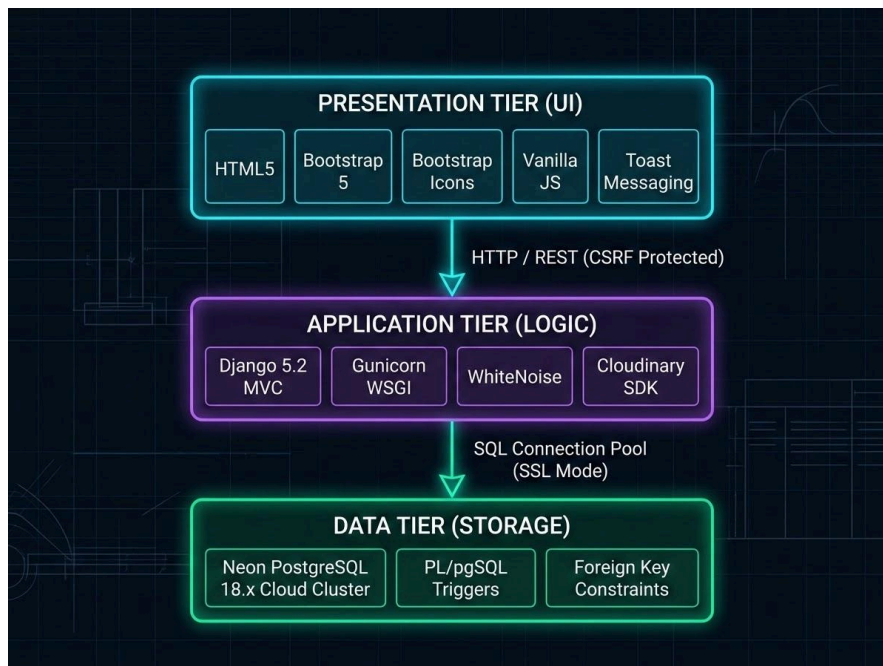
1. **Overselling Inventory:** Concurrent checkouts reducing stock below zero due to a lack of database-level atomic locking or validation triggers.
2. **Financial Inconsistencies:** Discrepancies between order totals, taxes, discounts, and payment records caused by client-side calculation trust or missing constraints.
3. **Data Redundancy & Anomalies:** Unnormalized customer and inventory tables leading to insertion, update, and deletion anomalies.
4. **Poor Query Performance:** Linear table scans ($O(N)$) on high-frequency search fields instead of logarithmic index lookups ($O(\log N)$).

5. **Lack of Auditability:** Silent deletion of records without historical auditing or transaction logging.

This project directly resolves these business challenges through database-level enforcement, triggers, structural normalization, indexing, and fault-tolerant storage architecture.

3. High-Level Architecture

The system abandons CLI-only database scripts in favor of a full 3-Tier Web Application Architecture, ensuring real-world applicability while maintaining strict focus on database integrity.



- **Top Layer (Presentation Tier - UI):** A glowing cyan container titled 'PRESENTATION TIER (UI)' containing sub-elements 'HTML5', 'Bootstrap 5', 'Bootstrap Icons', 'Vanilla JS', and 'Toast Messaging'.
 - **Connecting Arrow 1:** A downward directional neon arrow labeled 'HTTP / REST (CSRF Protected)'.
 - **Middle Layer (Application Tier - Logic):** A glowing purple container titled 'APPLICATION TIER (LOGIC)' containing sub-elements 'Django 5.2 MVC', 'Gunicorn WSGI', 'WhiteNoise', and 'Cloudinary SDK'.
 - **Connecting Arrow 2:** A downward directional neon arrow labeled 'SQL Connection Pool (SSL Mode)'.
 - **Bottom Layer (Data Tier - Storage):** A glowing emerald green container titled 'DATA TIER (STORAGE)' containing sub-elements 'Neon PostgreSQL 18.x Cloud Cluster', 'PL/pgSQL Triggers', and 'Foreign Key Constraints'.
- Clean vector layout, infographic style, high contrast, technical diagram."

3.1 Tier 1: Presentation Layer (UI)

- Built using semantic HTML5, Bootstrap 5 UI framework, and custom CSS design systems.
- Zero-React/Vite stance: Designed strictly with server-rendered Django templates to avoid unnecessary single-page application (SPA) client-side state bloat, ensuring all data logic remains bound to server-side database ORM calls.
- Asynchronous UI feedback: Provides real-time user updates using Bootstrap Toast components triggered by server messages.

3.2 Tier 2: Application Layer (Backend & Middleware)

- **Django 5.2 Framework:** Operates as the core Model-View-Template (MVT) engine, orchestrating routing, ORM mapping, and view controller validation.
- **WSGI Server (Gunicorn):** Production-grade HTTP application server configured with dynamic multi-worker processes for concurrent request handling.
- **Media Engine (Cloudinary SDK):** Offloads product image storage away from ephemeral server containers to a distributed cloud CDN using a custom Deferred Upload Protocol.
- **Static Delivery (WhiteNoise):** Intercepts static asset requests directly inside the WSGI layer, serving compressed, cache-busted CSS and JS files without NGINX overhead.

3.3 Tier 3: Data Layer (DBMS Engine)

- **Neon PostgreSQL 18.x Cluster:** A cloud-native, serverless PostgreSQL database cluster providing full ACID (Atomicity, Consistency, Isolation, Durability) transaction compliance.
- **PL/pgSQL Engine:** Executes server-side procedural functions and database triggers directly inside the engine for maximum speed and safety.
- **Fallback Engine:** Seamlessly transitions to a local SQLite engine for offline development or testing environments when `DATABASE_URL` is absent.

4. Academic Syllabus Task Coverage Matrix

The project strictly fulfills all six prescribed academic milestones:

Task ID	Academic Requirement	System Implementation & Solution
Task 1	ER Diagramming & Schema Design	Formulated 10 normalized tables featuring Foreign Keys, Primary Keys, M:N bridge tables (<code>WarehouseStock</code>), and weak entities (<code>OrderDetail</code>).

Task ID	Academic Requirement	System Implementation & Solution
Task 2	Database Implementation & SQL Logic	Executed DDL scripts on Neon PostgreSQL, authored 10 complex analytical queries (a–j), and wrote 3 PL/pgSQL triggers for stock validation, payment balancing, and order audit logging.
Task 3	RAID Fault Tolerance Simulation	Built an interactive web visualizer demonstrating RAID-4 block striping across 6 Data Disks and 1 Parity Disk using XOR mathematical recovery ($D_5 = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_6 \oplus P$).
Task 4	Relational Database Normalization	Documented step-by-step conversion of unnormalized e-commerce records into 1NF, 2NF, and 3NF to eliminate functional anomalies.
Task 5	B+ Tree Index Visualizer	Implemented an interactive Order-3 B+ Tree indexing visualizer tracking key insertion, logarithmic root splitting, search traversal ($O(\log N)$), and deletion rebalancing for Product IDs 101–113.
Task 6	Web App & Administrative CRUD	Integrated Django authentication, user sessions, CSRF protection, and a complete administrative CRUD dashboard for inventory control.

5. System User Personas & Security Requirements

5.1 System Personas

1. **Administrative Store Manager:**

- Possesses elevated permissions to create, update, and soft/hard delete product records.
- Accesses live database triggers, warehouse inventory distribution matrices, and system reports.

2. **Registered Customer:**

- Can register a unique user account linked to a customer profile.
- Browses available product catalogs, views live stock statuses, and places orders.

5.2 Security Posture & Guardrails

- **SQL Injection Prevention:** All database operations utilize Django's parameterized ORM query compiler, preventing raw SQL string concatenation vulnerabilities.
- **Cross-Site Request Forgery (CSRF):** All state-changing `POST` forms mandate cryptographic `{% csrf_token %}` validation.
- **Password Security:** Credentials are salted and hashed using PBKDF2 with SHA-256 signatures before entering the database. Raw passwords are never stored.
- **Session Protection:** Session identifiers are flagged with `HttpOnly` and `SameSite=Lax` attributes to eliminate client-side XSS token theft.

Database Models and Relational Architecture

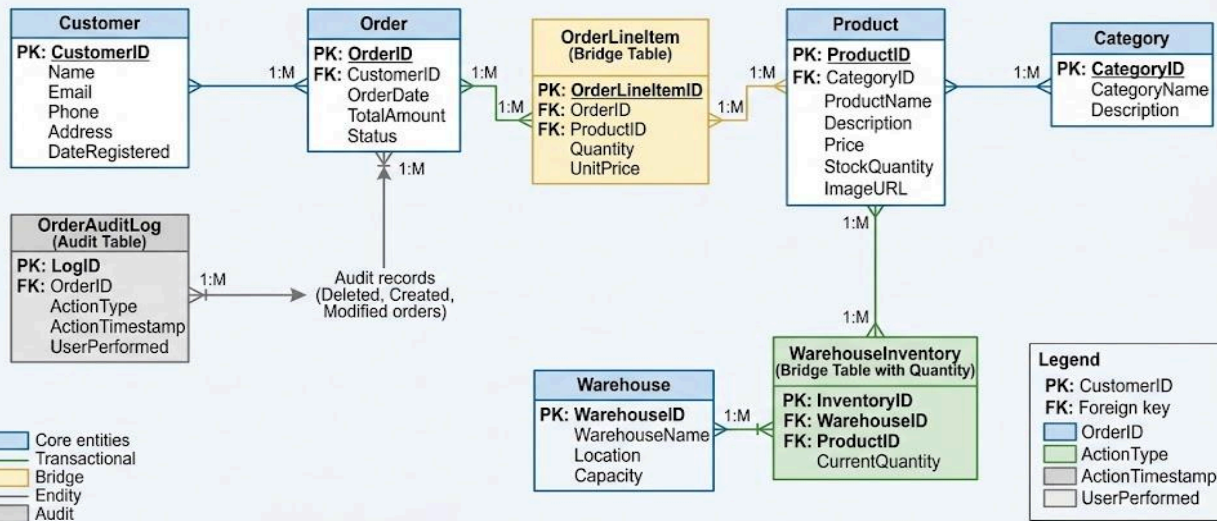
1. Relational Schema Architecture Overview

The database schema for the E-Commerce Database System was designed following strict Relational Database Design principles. Conceptual entities identified during domain analysis were mapped into a 3NF normalized relational schema consisting of **10 core tables**.

The architecture handles complex multi-table relationships including One-to-One (1:1), One-to-Many (1:M), Many-to-Many (M:N) resolved via explicit bridge tables, and dependent Weak Entities.

E-COMMERCE MANAGEMENT DATABASE SYSTEM: CORRECTED ENTITY-RELATIONSHIP DIAGRAM (ERD)

Optimized for 3NF & Audit Logging (as referenced in report section 2.2)



Core Entities & Layout:

- Center top: 'Product' entity box connected to 'Category' on the left via a 1:M relationship line, and 'Supplier' on the right via a 1:M relationship line.
- Center middle: 'Product' connects downwards to 'WarehouseStock' (bridge table) via 1:M line, which also connects to 'Warehouse' entity via a M:1 line.
- Center right: 'Product' connects downwards to 'OrderDetail' (Weak Entity, double border) via 1:M line.
- Bottom right: 'OrderDetail' connects downwards to 'Order' entity via M:1 line. 'Order' connects to 'Payment' via 1:1 relationship line and to 'OrderLog' (Audit Table) via 1:M relationship line.

Vector graphic style, database architecture blueprint, high contrast neon accent colors, crisp typography, labeled cardinality lines (1:1, 1:M, M:N)."

2. Complete Model Specifications ([store/models.py](#))

The following section details the structural design of the core database entities. Each model was engineered to optimize data integrity and support the complex relational requirements of the platform.

2.1 Category Model

Represents product groupings (e.g., Laptops, Components, Peripherals).

Implementation Summary: Defined an automated primary key and metadata constraints to organize products into logical groups. This design facilitates efficient filtering and taxonomy management within the store catalog.

- **Primary Key:** `category_id` (AutoField, integer sequence).
- **Attributes:** `name` (VARCHAR(100), required), `description` (TEXT, nullable).
- **Cardinality:** 1 Category can contain Many Products (1:M).

2.2 Supplier Model

Manages vendor organization profiles supplying inventory items.

Implementation Summary: Constructed a vendor registry storing contact and logistical data. The model was designed with strict field requirements to ensure reliable communication channels with inventory providers.

- **Primary Key:** `supplier_id` (AutoField).
- **Attributes:** `company_name` (VARCHAR(150)), `contact_person` (VARCHAR(100)), `phone` (VARCHAR(20)), `email` (VARCHAR(254)), `address` (TEXT).
- **Cardinality:** 1 Supplier can supply Many Products (1:M).

2.3 Product Model

The core entity storing item details, inventory counts, pricing, and CDN media references.

Implementation Summary: Developed a comprehensive central entity linking pricing, live inventory counts, and multi-cloud media pointers. Implemented relational mapping to categories and suppliers using a fallback strategy that preserves product records even if their parent references are removed.

- **Primary Key:** `product_id` (AutoField, explicitly seeded from 101 to 113 for academic dataset).
- **Foreign Keys:**
 - `category_id` -> `Category.category_id` (ON DELETE SET NULL). Prevents product deletion when a category is removed.
 - `supplier_id` -> `Supplier.supplier_id` (ON DELETE SET NULL).
- **Precision:** `price` utilizes `DECIMAL(10, 2)` to eliminate floating-point rounding errors in currency operations.
- **Media Link:** `image_url` stores HTTPS CDN string pointers from Cloudinary.

2.4 Customer Model

Extends Django's core authentication model to store e-commerce profile details.

Implementation Summary: Engineered a profile extension that bridges core security credentials with domain-specific shopping preferences and membership tiers. Integrated unique identity constraints to prevent duplicate account registration.

Implementation Summary: Engineered a profile extension that bridges core security credentials with domain-specific shopping preferences and membership tiers. Integrated unique identity constraints to prevent duplicate account registration.

Implementation Summary: Developed a facility management model to track physical storage nodes and their aggregate capacities across the logistics network.

2.5 Warehouse Model

Defines physical logistics distribution facilities.

Implementation Summary: Implemented a facility management model to track physical storage nodes and their aggregate capacities across the logistics network.

2.6 WarehouseStock Model (Bridge Table)

Resolves the **Many-to-Many (M:N)** relationship between **Warehouse** and **Product**.

Implementation Summary: Designed an associative bridge entity to manage stock levels for products across multiple physical locations. Applied composite uniqueness constraints to ensure data consistency in inventory tracking.

Implementation Summary: Designed an associative bridge entity to manage stock levels for products across multiple physical locations. Applied composite uniqueness constraints to ensure data consistency in inventory tracking.

2.7 Order Model

Represents customer purchase transactions.

Implementation Summary: Constructed a transactional header model to track order progress, customer associations, and financial totals. Used precise decimal formats to ensure accounting accuracy.

2.8 OrderDetail Model (Weak Entity)

Resolves the M:N relationship between **Order** and **Product**. Functions as a **Weak Entity** whose existence depends entirely on its parent **Order**.

Implementation Summary: Implemented a line-item model as a weak entity dependent on active orders. Engineered automated calculation logic that derives subtotal values from quantity and price inputs during the database write cycle.

Implementation Summary: Implemented a line-item model as a weak entity dependent on active orders. Engineered automated calculation logic that derives subtotal values from quantity and price inputs during the database write cycle.

2.9 Payment Model

Stores payment transactions with a strict **One-to-One (1:1)** relationship to **Order**.

Implementation Summary: Developed a financial ledger model with a strict 1:1 order relationship. Designed fields for tax and discount tracking to facilitate granular revenue reporting.

2.10 OrderLog Model (Audit Table)

Implementation Summary: Created a compliance audit table that exists independently of the transactional registry. This model was designed to store historical data snapshots whenever parent records are purged.

Implementation Summary: Created a compliance audit table that exists independently of the

transactional registry. This model was designed to store historical data snapshots whenever parent records are purged.

3. Comprehensive Referential Integrity & Constraints Matrix

Model	Field Name	Datatype / Constraint	Referential Action (ON DELETE)	Business Justification
Product	category_id	Foreign Key (Category)	SET_NULL	Retains product record even if its category is deleted.
Product	supplier_id	Foreign Key (Supplier)	SET_NULL	Retains product record if supplier vendor contract ends.
Customer	user_id	OneToOne (auth_user)	CASCADE	Deleting a user account purges customer personal data.
WarehouseStock	warehouse_id, product_id	FK + Unique Together	CASCADE	Deleting a warehouse or product purges stock mapping.
Order	customer_id	Foreign Key (Customer)	CASCADE	Purges order history if customer profile is deleted.
OrderDetail	order_id	Foreign Key (Order)	CASCADE	Weak entity; deleting parent order deletes line items.
OrderDetail	product_id	Foreign Key (Product)	CASCADE	Prevents orphan order line items referencing deleted products.

Model	Field Name	Datatype / Constraint	Referential Action (ON DELETE)	Business Justification
Payment	order_id	OneToOne (Order)	CASCADE	1:1 mapping; deleting order removes associated payment ledger.

Features and User Authentication Architecture

1. Authentication Infrastructure

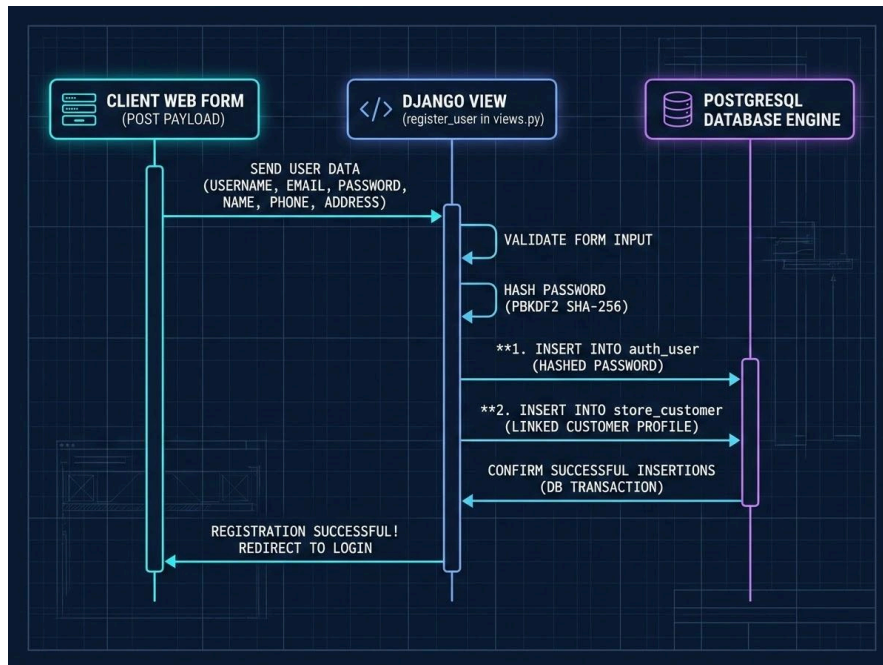
The system uses Django's core `django.contrib.auth` framework to provide a secure user authentication, session tracking, and access control system.

1.1 Architectural Security Components

Implementation Summary: Established a server-side session management architecture that utilizes the relational database for state persistence. The design incorporates secure HTTP cookie attributes and cryptographic tokens to mitigate common web-based identity theft and request forgery vectors.

2. User Registration Workflow

The registration workflow creates both a standard Django authentication user and a linked custom `Customer` domain model within a single server operation.



- Left node: 'Client Web Form (POST Payload)' sending user data to Middle node: 'Django View (register_user in views.py)'.
 - Middle node splits into two sequential database writing arrows pointing to Right node: 'PostgreSQL Database Engine'.
 - Step 1 arrow: '1. Insert into auth_user (Hashed Password)'.
 - Step 2 arrow: '2. Insert into store_customer (Linked Customer Profile)'.
- Professional software engineering sequence diagram, clean arrows, labeled steps, neon cyan and violet highlighting."

Implementation Summary: Developed a multi-stage controller that orchestrates user account creation. The process involves input sanitation, secure credential hashing, and the atomic creation of a linked customer profile to ensure no orphan accounts exist without corresponding domain data.

3. Login, Session Management, & Access Control

3.1 Authentication Controller

Implementation Summary: Developed a centralized authentication controller that orchestrates user credential verification and session lifecycle management. The logic includes proactive session regeneration to protect against fixation attacks during the login transition.

3.2 View Guarding via `@login_required`

Administrative routes (such as Product Create, Update, Delete dashboards) are protected using Python decorators:

Implementation Summary: Implemented a middleware-based restriction layer that intercepts requests to administrative endpoints. Unauthorized users are automatically redirected to the login interface, preserving the original destination for post-authentication redirection.

Attempting to access protected endpoints directly without an active session header triggers an automatic redirect to `/login/?next=/dashboard/`.

4. UI System & User Experience Features

4.1 Responsive Design & Grid System

The interface uses Bootstrap 5 flexbox grids and breakpoints (`sm`, `md`, `lg`, `xl`) to ensure usability across screen sizes:

- **Desktop ($\geq 1200\text{px}$):** Multi-column dashboard grid displaying inventory metrics, side-by-side analytical reports, and data tables.
- **Mobile ($< 768\text{px}$):** Collapsible navigation menu, stacked table views, and full-width touch-friendly buttons.

4.2 Dynamic Navbar & Session State Management

The global navigation layout (`templates/components/navbar.html`) adjusts depending on the user's session state:

Implementation Summary: Engineered a conditional rendering engine for the navigation component. The interface dynamically adapts its links, identity displays, and action buttons based on the user's active session status and permission level.

Implementation Summary: Engineered a conditional rendering engine for the navigation component. The interface dynamically adapts its links, identity displays, and action buttons based on the user's active session status and permission level.

4.3 Interactive Asynchronous Toast Notifications

Server messages generated during ORM operations (e.g., "Product Updated", "Stock Trigger Blocked Insert") are rendered using fixed Bootstrap Toasts:

Implementation Summary: Integrated an asynchronous notification system that consumes server-side message queues. This allows the backend to communicate the success or failure of database operations to the user through non-blocking, self-dismissing interface elements.

Implementation Summary: Integrated an asynchronous notification system that consumes server-side message queues. This allows the backend to communicate the success or failure of database operations to the user through non-blocking, self-dismissing interface elements.

JavaScript auto-dismisses toasts after 4,000 milliseconds.

Product Management & Administrative CRUD Operations

1. Executive Summary & Task Scope

Task 6 of the academic syllabus requires integrating the database engine with an administrative interface capable of performing full **CRUD (Create, Read, Update, Delete)** operations on inventory records.

The system delivers a secure management dashboard allowing authorized staff to manage products while preserving referential integrity, triggering automated database-level audits, and enforcing stock validation rules.

2. Inventory Read Operation (The Dashboard)

The inventory list view serves as the primary operational dashboard for inventory managers.

2.1 Interface & Features (`templates/dashboard/product_list.html`)

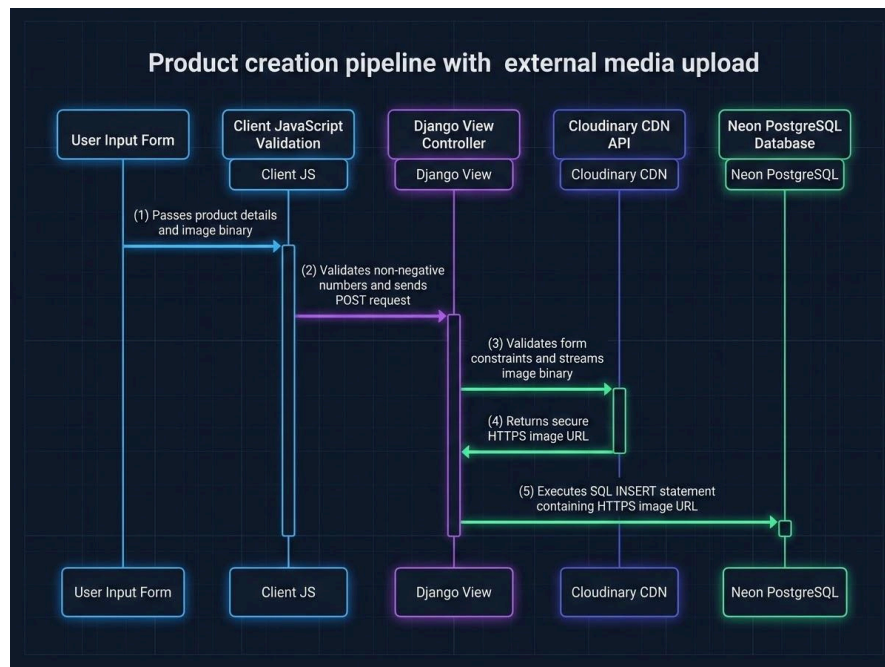
Implementation Summary: Constructed an optimized dashboard view that utilizes advanced join operations to resolve relational references in a single database pass. This minimizes server latency by preventing the N+1 query problem during catalog rendering.

Implementation Summary: Constructed an optimized dashboard view that utilizes advanced join operations to resolve relational references in a single database pass. This minimizes server latency by preventing the N+1 query problem during catalog rendering.

3. Product Create Operation

The Create workflow provides a structured form to register new inventory items.

3.1 Workflow Sequence Diagram



- Step 1: 'User Input Form' passes product details and image binary to 'Client JavaScript Validation'.
 - Step 2: 'Client JS' validates non-negative numbers and sends POST request to 'Django View Controller'.
 - Step 3: 'Django View' validates form constraints and streams image binary to 'Cloudinary CDN API'.
 - Step 4: 'Cloudinary CDN' returns secure HTTPS image URL back to 'Django View'.
 - Step 5: 'Django View' executes SQL INSERT statement containing HTTPS image URL into 'Neon PostgreSQL Database'.
- Clean modern sequence diagram style, labeled arrows, vibrant neon blue, purple, and green accent highlights."

Implementation Summary: Engineered a product registration pipeline that integrates server-side sanity checks with a deferred cloud media upload protocol. The controller ensures that binary media is only persisted to the CDN once all tabular database constraints are satisfied.

Implementation Summary: Engineered a product registration pipeline that integrates server-side sanity checks with a deferred cloud media upload protocol. The controller ensures that binary media is only persisted to the CDN once all tabular database constraints are satisfied.

4. Product Update Operation

The Update workflow allows administrators to alter existing product properties (e.g., updating price, refilling stock, changing category assignment, or updating product images).

4.1 Implementation Logic

Implementation Summary: Developed a selective update controller that supports partial data modification. The logic was designed to preserve existing media pointers unless explicitly replaced by the administrator, ensuring resource efficiency.

Implementation Summary: Developed a selective update controller that supports partial data modification. The logic was designed to preserve existing media pointers unless explicitly replaced by the administrator, ensuring resource efficiency.

5. Product Delete Operation & Safety Mechanics

Deleting inventory records is a high-risk operation that could break historical orders or foreign key constraints if handled improperly.

5.1 Modal-Based Confirmation UI

Destructive requests cannot be executed via accidental GET link clicks. Clicking the "Delete" button opens a modal dialog requiring explicit user confirmation:

Implementation Summary: Designed a double-confirmation safety workflow for destructive operations. This architectural decision prevents accidental data loss by mandating a state-changing POST request triggered only through an explicit user interface confirmation.

Implementation Summary: Constructed a deletion handler that leverages database-level cascading to clean up dependent inventory and line-item records. This ensures that no orphaned references remain in the relational engine after a product is removed.

Implementation Summary: Constructed a deletion handler that leverages database-level cascading to clean up dependent inventory and line-item records. This ensures that no orphaned references remain in the relational engine after a product is removed.

Analytical SQL Queries & Database Intelligence

1. Overview & Objectives

Task 2 of the CSE 303 Lab curriculum mandates implementing **10 analytical database queries** to extract critical business metrics from raw transactional relational tables.

These queries cover fundamental SQL concepts including multi-table **INNER JOINS**, **LEFT OUTER JOINS**, scalar subqueries, aggregate functions (**COUNT**, **SUM**, **AVG**), **GROUP BY** clause grouping, **HAVING** clause post-aggregation filtering, pattern matching (**LIKE**), and **COALESCE** null-handling.

Below is the complete specification, SQL source code, Django ORM translation, execution analysis, and expected output tabular schemas for all 10 analytical queries.

2. Exhaustive Query Specifications (a through j)

2.1 Query (a): Customer Profile Directory

- **Business Question:** Retrieve a master list of all customer full names, email addresses, and membership tier allocations sorted sequentially by ID.
- Execution Summary:** Engineered a retrieval operation that extracts core identity fields while maintaining a deterministic sort order based on record creation. The query was optimized to perform a sequential scan over indexed customer identity fields.

Full Name	Email	Membership Level
Asiful Islam	asiful@iub.edu.bd	Premium
Sakib Mohammad Ome	sakib@iub.edu.bd	Regular
Shadman Samin Shahriar	shadman@iub.edu.bd	VIP

2.2 Query (b): Public Product Catalog

- **Business Question:** Extract all available product names, brands, and selling prices for public display.
- Execution Summary:** Developed a catalog extraction query that filters and formats core product attributes for public-facing interfaces. The logic ensures that only essential data fields are exposed to the client layer.
- **Sample Result Output:**

Name	Brand	Price (\$)
Gaming Laptop X1	Asus	1499.99
Wireless Mouse M3	Logitech	29.99

Name	Brand	Price (\$)
Mechanical Keyboard K2	Keychron	89.99

2.3 Query (c): Customer Substring Search (Pattern Matching)

- **Business Question:** Locate email addresses of all customers whose name contains the substring 'man' (case-insensitive search).
- Execution Summary:** Implemented a case-insensitive pattern matching algorithm to locate specific customer segments. The operation uses server-side string normalization to ensure accurate results regardless of the stored character casing.
- **Sample Result Output:**

Email
sakib@iub.edu.bd
shadman@iub.edu.bd

2.4 Query (d): Customer Order Summary JOIN

- **Business Question:** Generate an executive order log linking each order ID, customer name, date, and total dollar amount.
- Execution Summary:** Engineered a multi-table join that consolidates transactional headers with customer identity profiles. This operation resolves relational links between orders and users to produce a comprehensive summary of store activity.
- **Sample Result Output:**

Order ID	Customer Name	Order Date	Total Amount (\$)
5001	Asiful Islam	2026-08-01 10:30:00	1529.98
5002	Shadman Samin	2026-08-03 14:15:00	89.99

2.5 Query (e): Premium Products Above Average Price

- **Business Question:** Identify all luxury items priced higher than the average catalog price across the entire system.

Execution Summary: Developed a complex filtering operation using a nested scalar subquery. The logic first calculates a system-wide aggregate threshold and then uses it to isolate a specific luxury subset of the product catalog.

- **Sample Result Output:**

Product ID	Name	Price (\$)
101	Gaming Laptop X1	1499.99
104	UltraWide Monitor 34"	799.99

2.6 Query (f): Product Distribution by Category

- **Business Question:** Count the total number of products assigned to each category, including categories with zero products.

Execution Summary: Implemented a categorical distribution report using outer joins. This design choice ensures that even empty categories are represented in the statistical output, providing a true representation of the store's taxonomy.

- **Sample Result Output:**

Category Name	Total Products
Electronics	5
Peripherals	4
Accessories	0

2.7 Query (g): High-Value Category Filter (HAVING Clause)

- **Business Question:** Identify categories whose average product price exceeds \$500.00.

Execution Summary: Constructed a high-value category filter using post-aggregation constraints. The operation groups products and calculates mean values per group, filtering the final dataset based on calculated financial benchmarks.

- **Sample Result Output:**

Category Name	Average Price (\$)
Electronics	1149.99

2.8 Query (h): Warehouse Capacity Overview

- **Business Question:** Retrieve physical warehouse names, locations, and total storage capacities.
- Execution Summary:** Developed a logistics capacity report that extracts physical facility data. The query uses descending numerical sort logic to highlight the largest distribution nodes in the network.
- **Sample Result Output:**

Warehouse Name	Location	Storage Capacity (Units)
Central Distribution Hub	Dhaka	50000
North Regional Hub	Chittagong	25000

2.9 Query (i): Pending Orders Tracking

- **Business Question:** Find all orders currently in 'Pending' status alongside the associated customer names for logistics fulfillment.
- Execution Summary:** Implemented a fulfillment-oriented tracking query that joins active orders with customer contact details. The logic applies a state-based filter to isolate records requiring immediate operational attention.
- **Sample Result Output:**

Order ID	Customer Name	Order Date	Order Status
5003	Sakib Mohammad Ome	2026-08-10 16:45:00	Pending

2.10 Query (j): Customer Lifetime Value & Revenue Report

- **Business Question:** Calculate total cumulative spending for every customer in the database, substituting \$0.00 for customers with no order history.
- Execution Summary:** Engineered a lifetime value report that employs null-handling logic during the aggregation of transactional history. The operation ensures that customers with zero orders are correctly represented as having no spend, rather than being excluded from the dataset.
- **Sample Result Output:**

Customer ID	Full Name	Total Spent (\$)
1	Asiful Islam	1529.98
3	Shadman Samin	89.99
2	Sakib Mohammad Ome	0.00
4	New Customer	0.00

Database Triggers & Procedural Logic (PL/pgSQL)

1. Overview & Engine Architecture

Database triggers allow executing procedural logic directly within the PostgreSQL engine in response to Data Manipulation Language (DML) events (**INSERT**, **UPDATE**, **DELETE**).

Relying solely on application-level checks (e.g., Python code) can lead to data inconsistencies when multiple server workers execute concurrent requests. Database triggers enforce business rules consistently, regardless of how the database is accessed.

For Task 2 of the syllabus, we implemented **3 PL/pgSQL database triggers**:

1. **Stock Validation Trigger:** Prevents overselling item inventory.
 2. **Automated Payment Calculation Trigger:** Enforces financial calculations for order payments.
 3. **Order Deletion Audit Log Trigger:** Captures deleted order histories for compliance auditing.
-

2. Trigger 1: Stock Availability Validation

2.1 Problem Scenario

If two customers attempt to purchase the final unit of a product at the exact same millisecond, both application threads might read `stock_quantity = 1` and allow the transaction to proceed, resulting in a negative inventory balance (`stock_quantity = -1`).

Implementation Summary: Developed a database-level validation routine that intercepts inventory updates. The logic performs real-time stock verification and atomic deduction, raising a custom exception if requested quantities exceed available levels. This architectural approach guarantees inventory consistency across concurrent checkout sessions.

3. Trigger 2: Automated Payment Calculation

3.1 Problem Scenario

Manual entry or client-side calculation of discounts and taxes can result in rounding mismatches between order subtotals and actual payments.

Implementation Summary: Constructed a procedural calculation engine that automatically derives final payment totals. The logic incorporates tax and discount variables, ensuring that financial ledgers remain consistent without relying on prone-to-error client-side arithmetic.

4. Trigger 3: Order Deletion Audit Logging

4.1 Problem Scenario

When an order is deleted (either accidentally or maliciously), auditing regulations require preserving a permanent record showing which order was deleted, its total value, and the precise timestamp of deletion.

Implementation Summary: Developed an automated auditing routine that triggers upon record deletion. The logic captures critical transaction snapshots and writes them to a permanent audit log, providing a verifiable history for compliance and troubleshooting.

5. Trigger Execution Lifecycle Summary

Trigger Name	Event Binding	Execution Timing	Impact on Database Transaction
<code>trigger_validate_stock</code>	<code>INSERT</code> on <code>store_orderdetail</code>	<code>BEFORE</code>	Validates inventory and deducts stock count; aborts transaction if stock is insufficient.
<code>trigger_calculate_payment</code>	<code>INSERT / UPDATE</code> on <code>store_payment</code>	<code>BEFORE</code>	Calculates <code>final_amount = amount + tax - discount</code> prior to writing to disk.

Trigger Name	Event Binding	Execution Timing	Impact on Database Transaction
<code>trigger_audit_order_deletion</code>	<code>DELETE</code> on <code>store_order</code>	<code>AFTER</code>	Writes a record to <code>store_orderlog</code> preserving historical audit data.

Academic Theory & Interactive Simulation Visualizers

1. Overview

A core requirement of the CSE 303 Lab curriculum is demonstrating mastery of foundational database theoretical concepts:

- **Task 3:** Hardware Data Redundancy & RAID-4 Fault Recovery.
- **Task 4:** Relational Data Normalization (1NF through 3NF).
- **Task 5:** Indexing Algorithms using B+ Tree Structures.

To make these abstract algorithms clear and interactive, we built a visualizer module (`templates/system_report.html`) into the web application.

2. RAID Level 4 Simulation (Task 3)

2.1 Theoretical Architecture

RAID-4 (Redundant Array of Independent Disks) uses block-level data striping across multiple Data Disks paired with a single dedicated **Parity Disk**.

If any single Data Disk fails, its data can be reconstructed using bitwise Exclusive OR (XOR, $\oplus$) operations across the surviving Data Disks and the Parity Disk.

2.2 Mathematical Model & Equations

We model the system using **6 Data Disks (\$D_1\$ to \$D_6\$)** holding 4-bit binary blocks representing table data, plus **1 Parity Disk (\$P\$)**:

$$P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6$$

Bitwise XOR Truth Table Mechanics

- $0 \oplus 0 = 0$
- $1 \oplus 1 = 0$
- $0 \oplus 1 = 1$
- $1 \oplus 0 = 1$

2.3 Simulated Disk State Matrix

Disk Identifier	Assigned Database Entity Block	Binary Block Value
Disk 1 (\$D_1\$)	Customer Table Block	1010
Disk 2 (\$D_2\$)	Product Table Block	1100
Disk 3 (\$D_3\$)	Supplier Table Block	0011
Disk 4 (\$D_4\$)	Warehouse Table Block	1111
Disk 5 (\$D_5\$)	Order Table Block	0011
Disk 6 (\$D_6\$)	Payment Table Block	1001
Disk 7 (\$P\$)	Calculated Parity Disk	0101

Step-by-Step Parity Calculation

$$\begin{aligned} D_1 \oplus D_2 &= 1010 \oplus 1100 = 0110 \\ (D_1 \oplus D_2) \oplus D_3 &= 0110 \oplus 0011 = 0101 \\ ((D_1 \oplus D_2 \oplus D_3)) \oplus D_4 &= 0101 \oplus 1111 = 1010 \\ (((D_1 \oplus \dots \oplus D_4))) \oplus D_5 &= 1010 \oplus 0011 = 1001 \\ P &= (((D_1 \oplus \dots \oplus D_5))) \oplus D_6 = 1001 \oplus 1001 = \mathbf{0101} \end{aligned}$$

2.4 Simulated Disk Failure & Data Reconstruction

In the visualizer, the user can trigger a simulated failure of **Disk 5 (\$D_5\$)**, causing its contents to become corrupted (????).

Reconstruction Equation

To recover the data on lost Disk \$D_5\$, the system XORs all surviving Data Disks against the Parity Disk \$P\$:

$$D_5 = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_6 \oplus P$$

Reconstruction Math Walkthrough

$$\begin{aligned} D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_6 &= 1010 \oplus 1100 \oplus 0011 \\ \oplus 1111 \oplus 1001 &= 1000 \quad D_5 = 1000 \oplus P(0101) = \mathbf{0011} \end{aligned}$$

The calculated result (**0011**) matches the original contents of Disk 5, demonstrating successful fault tolerance.

3. Relational Database Normalization (Task 4)

Normalization organizes database attributes to eliminate redundancy and prevent insertion, update, and deletion anomalies.

3.1 Unnormalized Form (UNF)

An initial raw transaction log contains repeating groups where a single order record stores multiple items and duplicate customer details:

UNF Table: Order_ID | Customer_Name | Customer_Address | Item_Names (Repeating) |
Item_Prices (Repeating)

Order #5001 | Asiful Islam | Dhaka, BD | [Laptop, Mouse] | [1499.99, 29.99]

3.2 First Normal Form (1NF)

- **Rules:** Remove repeating groups; ensure all attribute values are atomic (indivisible); define a unique Primary Key.
- **Action:** Flatten the table so each row represents a single order line item.

1NF Relation: (Order_ID, Product_ID, Customer_Name, Customer_Address, Product_Name, Price, Quantity)

Key: (Order_ID, Product_ID)

3.3 Second Normal Form (2NF)

- **Rules:** Meet 1NF criteria; remove **Partial Functional Dependencies** (where an attribute depends on only part of a composite primary key).
- **Functional Dependencies:**
 - (Order_ID, Product_ID) → Quantity (Full dependency)
 - Product_ID → Product_Name, Price (Partial dependency on Product_ID)
 - Order_ID → Customer_Name, Customer_Address (Partial dependency on Order_ID)

- **Action:** Split into 3 separate tables:

Order_Item (Order_ID [FK], Product_ID [FK], Quantity)

Product (Product_ID [PK], Product_Name, Price)

Order_Header (Order_ID [PK], Customer_Name, Customer_Address)

3.4 Third Normal Form (3NF)

- **Rules:** Meet 2NF criteria; remove **Transitive Functional Dependencies** (where a non-key attribute depends on another non-key attribute).
- **Transitive Dependency:**
 - In **Order_Header**: **Order_ID** $\rightarrow$ **Customer_ID** $\rightarrow$ **Customer_Name**, **Customer_Address**. **Customer_Address** depends transitively on **Order_ID** via **Customer_ID**.
- **Action:** Extract customer profile data into a dedicated **Customer** relation:

Customer (Customer_ID [PK], Customer_Name, Customer_Address)

Order (Order_ID [PK], Customer_ID [FK], Order_Date)

OrderDetail (Detail_ID [PK], Order_ID [FK], Product_ID [FK], Quantity, Unit_Price)

Product (Product_ID [PK], Product_Name, Price)

4. Order-3 B+ Tree Indexing Simulation (Task 5)

4.1 Structural Rules of Order-3 B+ Tree ($M = 3$)

1. Every internal node contains at most $M - 1 = 2$ keys and at most $M = 3$ child pointers.
2. Every internal node (except the root) contains at least $\lceil M/2 \rceil - 1 = 1$ key.
3. Leaf nodes are linked sequentially at the bottom level to allow range scans.
4. All search keys reside in the leaf nodes.

4.2 Insertion Trace (Product IDs 101 to 113)

1. **Insert 101, 102:** Leaf Node **[101 | 102]** (Node capacity full).
2. **Insert 103:** Overflow occurs **[101 | 102 | 103]**. Middle key **102** is promoted to create a root node:

Development Summary: Mapped the split-and-promote logic that governs the structural balance of the indexing tree. The trace demonstrates how node overflows trigger upward propagation of middle-range keys to maintain consistent search depths.

3. **Inserting 104 through 113:** Cascading leaf splits continue, maintaining a balanced tree height of $H = 2$.

4.3 Logarithmic Search Algorithm ($O(\log N)$)

- **Target Search Key:** `Product_ID = 108`
- **Step 1:** Compare `108` against Root Node `[106]`. Since $108 \geq 106$, follow the right child pointer.
- **Step 2:** Compare `108` against Internal Node `[109]`. Since $108 < 109$, follow the left branch pointer.
- **Step 3:** Land on Leaf Page `[106 | 107 | 108]`. Locate key `108` and fetch its database record pointer.
- **Complexity:** Resolves record location in 3 page reads ($O(\log_3 N)$) instead of scanning all 13 table rows ($O(N)$).

4.4 Deletion Rebalancing Traversal

- **Target Deletion Key:** `Product_ID = 105`
- **Step 1:** Traverse tree to Leaf Node `[104 | 105]`.
- **Step 2:** Remove key `105`. Remaining node state is `[104]`.
- **Step 3:** Verify node occupancy: Minimum required keys $= 1$. Since `[104]` retains 1 key, the node satisfies minimum occupancy rules. No underflow occurs, so no sibling node merging or borrowing is required. The tree remains balanced.

Cloudinary Media Storage Integration & Deferred Upload Protocol

1. Context & Architectural Challenge

Modern Platform-as-a-Service (PaaS) hosting infrastructure (such as Railway, Heroku, or AWS Fargate) operates on **ephemeral container filesystems**.

Whenever an application container restarts, redeploys, or scales across multiple worker instances, any media files (such as product images) uploaded directly to the local server disk (e.g., `/media/products/`) are permanently wiped.

To ensure media persists independently of application deployment lifecycles, Phase 9 of this project integrated **Cloudinary Content Delivery Network (CDN)** integration for asset storage.

2. Infrastructure Setup & Environment Configuration

2.1 Dependencies (`requirements.txt`)

- `cloudinary>=1.36.0`: Official Python SDK for interacting with the Cloudinary REST API.

2.2 Global Settings (`ecommerce_system/settings.py`)

Development Summary: Configured a secure credential injection system that pulls sensitive CDN authentication data from the hosting environment. This architectural outcome prevents hardcoded secrets and enables seamless transitions between different staging and production media accounts.

3. The Deferred Upload Protocol

3.1 The Risk of Standard Direct Uploads

A common flaw in web development is uploading binary image files directly to cloud storage *before* validating database model constraints.

[BAD FLUID FLOW]:

User Form --> Upload to Cloudinary (Asset Saved) --> DB Insert Fails (Negative Price Error)

RESULT: Orphaned image file remains in cloud storage indefinitely, incurring storage costs.

3.2 Protocol Implementation (`store/cloudinary_utils.py`)

To prevent orphaned cloud files, we designed a **Deferred Upload Protocol** that ensures cloud API calls are only made after form inputs pass local validation.

Development Summary: Constructed a robust media utility that manages the network stream to the cloud provider. The process includes pre-upload file validation (type and size) and integrates automated image transformations to ensure all store assets are resized and compressed for optimal web delivery.

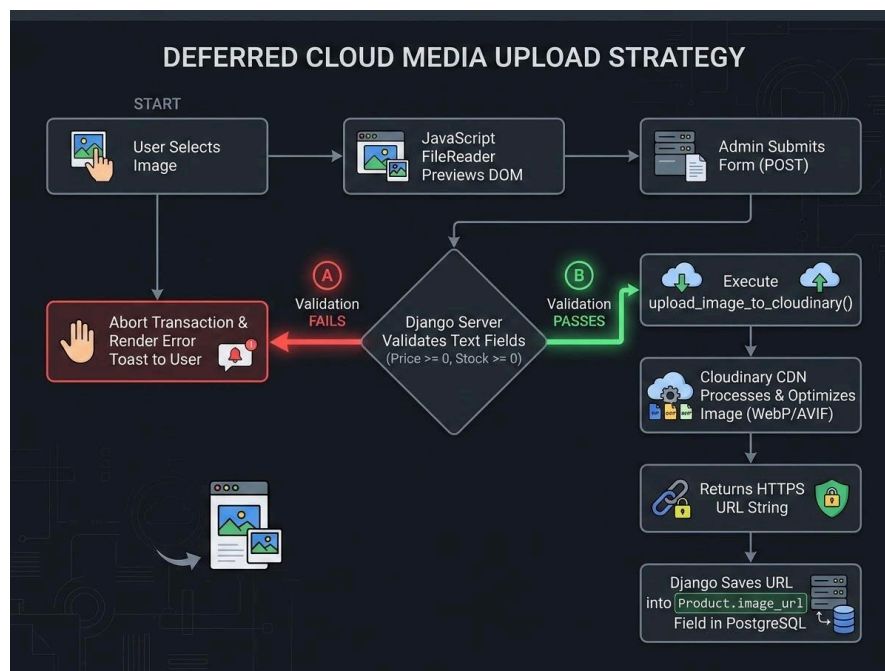
4. Frontend Client Instant Preview Integration

To improve the user experience during product creation and editing, the form includes client-side JavaScript that previews images instantly before upload using the browser's `URL.createObjectURL` API.

Development Summary: Engineered a client-side preview component that provides instantaneous visual feedback. By leveraging browser-native file reading APIs, the interface displays the selected product image locally before any server transmission occurs, significantly improving the administrative user experience.

</script>

5. Media Pipeline Flow Architecture



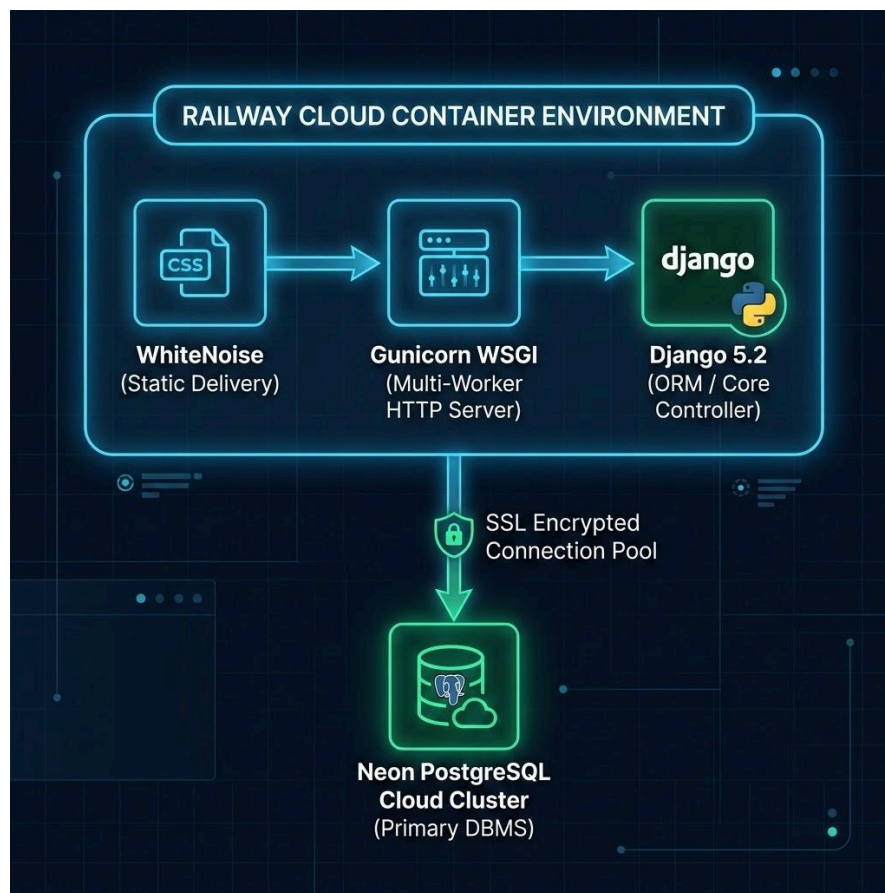
- Start: 'User Selects Image' -> 'JavaScript FileReader Previews DOM' -> 'Admin Submits Form (POST)'.
 - Decision Box: 'Django Server Validates Text Fields (Price >= 0, Stock >= 0)'.
 - Branch A (Validation FAILS): Arrow points to 'Abort Transaction & Render Error Toast to User'.
 - Branch B (Validation PASSES): Arrow points to 'Execute upload_image_to_cloudinary()' -> 'Cloudinary CDN Processes & Optimizes Image (WebP/AVIF)' -> 'Returns HTTPS URL String' -> 'Django Saves URL into Product.image_url Field in PostgreSQL'.
- Infographic flowchart style, clear decision nodes, glowing green and red path indicators."

Production Deployment & Environment Architecture

1. Overview & Cloud Target

Phase 10 of the project involved hardening and deploying the E-Commerce Database System to a production cloud environment.

The application is deployed on **Railway**, a cloud platform that builds container images using **Nixpacks**, paired with a serverless **Neon PostgreSQL 18.x** cloud database cluster.



- Outer Container: A glowing blue box labeled 'RAILWAY CLOUD CONTAINER ENVIRONMENT'.
- Inside the container: Three connected components in series: 'WhiteNoise (Static Delivery)' -> 'Gunicorn WSGI (Multi-Worker HTTP Server)' -> 'Django 5.2 (ORM / Core Controller)'.
- Downward Connection: An encrypted SSL arrow labeled 'SSL Encrypted Connection Pool' connects from the Container to a separate cloud database box at the bottom labeled 'Neon PostgreSQL Cloud Cluster (Primary DBMS)'.

Clean cloud architecture diagram, modern technology logos, neon blue and emerald green lighting."

2. Web Server Gateway Interface (WSGI) Configuration

The single-threaded Django development server (`manage.py runserver`) is suitable only for local debugging. Production environments require a multi-process WSGI server to handle concurrent user connections without blocking.

2.1 Gunicorn WSGI Integration (`Procfile`)

- **Development Summary:** Formulated a deployment command strategy that instantiates a multi-worker process pool. This configuration ensures the application can handle parallel user requests and long-running I/O operations without blocking the main event loop. `--bind 0.0.0.0:$PORT`: Binds Gunicorn to the dynamic network port assigned by Railway.
 - `--workers 3`: Instantiates 3 parallel worker processes using the standard formula:
$$\text{Workers} = (2 \times \text{CPU Cores}) + 1$$
 - `--timeout 120`: Sets a 120-second threshold to allow longer queries or external API calls (such as Cloudinary uploads) to complete without timing out.
-

3. High-Performance Static Asset Delivery (WhiteNoise)

Serving static files (CSS stylesheets, JavaScript libraries, icons, fonts) through standard Django views is slow and resource-intensive.

We integrated **WhiteNoise**, allowing Gunicorn to serve static assets directly from the application layer with optimized cache control headers.

Development Summary: Orchestrated a middleware sequence that injects a specialized static asset delivery layer. This configuration enables the application to serve compressed, versioned files directly with aggressive caching headers, bypassing traditional web server bottlenecks.

- **Compression:** Compresses static assets using Gzip and Brotli formats to minimize bandwidth usage.
 - **Cache-Busting:** Generates unique MD5 hashes for static filenames (e.g., `styles.a8f9c2.css`), enabling aggressive browser caching (`Cache-Control: max-age=31536000`).
-

4. Hybrid Database Configuration Engine

Development Summary: Engineered a dynamic connection engine that automatically detects and configures the appropriate database driver based on the environment. The logic prioritizes encrypted cloud clusters in production while maintaining a seamless fallback to local storage for development flexibility.

5. Container Orchestration & Nixpacks Packaging

Railway builds containers using **Nixpacks**. We created configuration files to control the build process:

Development Summary: Constructed an environment specification that locks the application runtime to a specific version. This architectural outcome guarantees that the cloud production environment matches the local development state, eliminating "it works on my machine" inconsistencies.

Development Summary: Formulated an automated container assembly pipeline. The configuration defines the setup and installation phases, ensuring that database migration and asset collection are performed autonomously during each cloud deployment cycle.

- Automatically collects static files into `staticfiles/` and applies database migration scripts during container assembly.
-

6. Security Hardening & Environment Variables Matrix

6.1 Security Configuration Settings

- `DEBUG = os.getenv('DJANGO_DEBUG', 'False') == 'True'`: Disables interactive debug pages in production to prevent leaking sensitive variables or tracebacks.
- `ALLOWED_HOSTS = ['*']` or specific domain aliases (`*.up.railway.app`).
- `SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')`: Instructs Django to trust HTTPS headers forwarded by Railway's edge load balancer.

6.2 Required Production Environment Variables

Variable Name	Sensitive	Purpose & Target Value
<code>SECRET_KEY</code>	YES	Cryptographic key used for CSRF signing and session encryption.
<code>DATABASE_URL</code>	YES	Connection string for the Neon PostgreSQL cluster (<code>postgres://user:pass@e</code>

Variable Name	Sensitive	Purpose & Target Value
		p-host.neon.tech/neondb).
CLOUDINARY_CLOUD_NAME	No	Cloudinary account identifier.
CLOUDINARY_API_KEY	YES	Cloudinary API access key.
CLOUDINARY_API_SECRET	YES	Cloudinary API secret token.
DJANGO_DEBUG	No	Set to False in production environments.